

Ejercicio Práctico: Buscador de Pokémon con Fetch API

Objetivo

Construir una aplicación web sencilla que permita consultar los datos de un Pokémon ingresando su nombre o ID numérico, mostrando en pantalla su nombre oficial y su imagen frontal. Deberán implementarse dos variantes de la misma consulta para comparar el manejo asíncrono con `.then` y con `async/await`.

Endpoint a utilizar

- **URL:** `https://pokeapi.co/api/v2/pokemon/{nombre_o_id}`
- **Método HTTP:** GET
- **Ejemplo de consulta:** `https://pokeapi.co/api/v2/pokemon/pikachu` o `https://pokeapi.co/api/v2/pokemon/25`

Datos a extraer del JSON de respuesta:

- Nombre: `data.name`
- Imagen frontal: `data.sprites.front_default`

Estructura HTML de base

Crear un archivo `index.html` con la siguiente estructura básica:

```
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>Buscador de Pokémon</title>
```

```
</head>
```

```
<body>
```

```
<h1>Buscador PokéAPI</h1>
```

```
<input type="text" id="pokemonInput" placeholder="Nombre o ID (ej: ditto, 1)">
```

```
<button id="btnThen">Buscar con .then</button>
```

```
<button id="btnAsync">Buscar con async/await</button>

<div id="resultado" style="margin-top: 20px;">

  <h2 id="pokemonNombre"></h2>

  <img id="pokemonImagen" src="" alt="" style="display: none; width: 150px;">

  <p id="mensajeError" style="color: red;"></p>

</div>

<script src="app.js"></script>

</body>

</html>
```

Requerimientos de JavaScript (`app.js`)

- Captura y normalización de datos:**
 - Obtener el valor ingresado en el campo de texto y convertirlo a minúsculas (`.toLowerCase()`) antes de armar la URL.
 - Si el campo está vacío, advertir al usuario y no disparar la petición.
- Implementación A: Petición con `.then()`**
 - Crear la función `buscarConThen()` vinculada al evento de click de `#btnThen`.
 - Realizar la llamada con `fetch()`.
 - Verificar la propiedad `response.ok`. Si es falsa (ej. código 404 por Pokémon inexistente), lanzar un error con `throw new Error(...)`.
 - Procesar el JSON mediante la promesa de `.json()`.
 - Enviar los datos obtenidos a la función encargada de pintar el DOM.
 - Capturar cualquier fallo con `.catch()` y mostrar el mensaje en el elemento `#mensajeError`.
- Implementación B: Petición con `async / await`**
 - Crear la función `async function buscarConAsync()` vinculada al click de `#btnAsync`.
 - Implementar un bloque `try / catch` para controlar el flujo y capturar excepciones.
 - Pausar la ejecución con `await` para la llamada `fetch()` y para el parsing de `response.json()`.
 - Validar igualmente si `response.ok` es verdadero o falso antes de parsear.
 - En caso de éxito, enviar el objeto a la función de renderizado; en caso de error, atraparlo en el `catch` y reportarlo en pantalla.

4. Renderizado en el DOM:

- Crear una función auxiliar (ejemplo: `function mostrarPokemon(datos)`) que actualice el texto de `#pokemonNombre`, asigne la URL a `#pokemonImagen.src`, haga visible la imagen y limpie los mensajes de error previos.

Restricciones de código

- Prohibido el uso de `var`: utilizar únicamente `let` y `const`.
- Prohibido el uso de arrow functions (`=>`): declarar todas las funciones y callbacks con la sintaxis tradicional (`function nombre(...) { ... }` o `function(...) { ... }`).
- Limpiar la imagen y el nombre anterior cada vez que se inicie una nueva búsqueda o ocurra un error.